SQL PROJECT / PORTFOLIO

PIZZA SALES ANALYSIS



Turning sales data into meaningful business insights

BUILT WITH SQL

Presented by Nikhil Dhyani

Business questions • SQL queries • Sales analysis

Hello, I'm Nikhil

In this project, I used SQL to analyze pizza sales data and answer 13 business questions. The analysis explores order volume, revenue, popular pizza types and sizes, category performance, and sales patterns over time.



PROJECT FOCUS

Explore pizza sales data and answer practical business questions.

ANALYTICAL GOAL

Identify patterns in orders, product demand, and revenue.

OUTCOME

Practice SQL and communicate findings clearly.

Project at a glance

13 BUSINESS QUESTIONS

A structured set of questions about orders, products, and revenue.

4 CORE TABLES

Orders, order details, pizzas, and pizza types (standard schema; verify against your database).

SQL TECHNIQUES

Joins, aggregation, grouping, date/time analysis, ranking, and running totals.

Project workflow

01 Understand the data

02 Write SQL queries

03 Compare sales patterns

04 Summarize insights

Dataset schema

Standard Pizza Sales dataset structure — confirm exact columns and relationships against the supplied database.



PK = Primary Key • **FK = Foreign Key**

Join path: orders → order_details → pizzas → pizza_types

Schema is illustrative because the uploaded SQL file does not include CREATE TABLE statements.

SQL techniques used

JOINS

Combine related tables to connect orders, pizzas, and categories.

AGGREGATE FUNCTIONS

SUM(), COUNT(), AVG(), and revenue calculations.

GROUP BY & ORDER BY

Summarize results and sort products by quantity or revenue.

DATE & TIME ANALYSIS

Study order patterns by date and hour.

WINDOW FUNCTIONS

RANK() and partitioned ranking for top pizzas by category.

SUBQUERIES

Build a ranked result from an aggregated revenue dataset.

Business questions | Orders & revenue

01	Retrieve the total number of orders placed.	COUNT / distinct order count
02	Calculate the total revenue generated from pizza sales.	SUM(quantity × price)
03	Identify the highest-priced pizza.	Price comparison / ordering
04	Identify the most common pizza size ordered.	Quantity aggregation by size
05	List the top 5 most ordered pizza types along with their quantities.	GROUP BY + ORDER BY + LIMIT

Business questions | Categories & time

06	Join the necessary tables to find the total quantity of each pizza category ordered.	JOIN + SUM + GROUP BY
07	Determine the distribution of orders by hour of the day.	Extract hour + COUNT
08	Join relevant tables to find the category-wise distribution of pizzas.	JOIN + category grouping
09	Group the orders by date and calculate the average number of pizzas ordered per day.	Date grouping + AVG

Business questions | Revenue ranking

10	**Determine the top 3 most ordered pizza types based on revenue.**	Rank pizza types by revenue
11	**Calculate the percentage contribution of each pizza type to total revenue.**	Revenue share percentage
12	**Analyze the cumulative revenue generated over time.**	Running total / window function
13	**Determine the top 3 most ordered pizza types based on revenue for each pizza category.**	RANK() OVER (PARTITION BY)

Total orders

Question 1: Retrieve the total number of orders placed.

```sql
SELECT
  COUNT(order_id) AS total_orders
FROM orders;
```



WHAT IT DOES

COUNT(DISTINCT order_id) counts each order once. This example assumes the standard dataset's orders table contains one row per order.

Illustrative SQL example — replace with your exact query if your project uses different SQL.

Total revenue

Question 2: Calculate the total revenue generated from pizza sales.

```sql
SELECT
    ROUND(
        SUM(order_details.quantity * pizzas.price),
        2
    ) AS total_sales
FROM
order_details
JOIN
pizzas
    ON pizzas.pizza_id = order_details.pizza_id;
```



WHAT IT DOES

Each line item's revenue is quantity multiplied by pizza price. SUM() adds those line-item values together.

Illustrative SQL example — assumes the standard Pizza Sales table and column names.

Top 5 pizza types

Question 5: List the top 5 most ordered pizza types along with their quantities.

```sql
SELECT
    pizza_types.name,
    SUM(order_details.quantity) AS quantity
FROM
pizza_types
JOIN
pizzas
    ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN
order_details
    ON order_details.pizza_id = pizzas.pizza_id
GROUP BY
pizza_types.name
ORDER BY
quantity DESC
LIMIT
5;
```



WHAT IT DOES

The joins connect each order line to a pizza and its type. The query totals quantity per type and returns the five highest totals.

Illustrative SQL example — compare it with the exact query you wrote.

Jan 2015 – Dec 2015

Cumulative revenue

Question 12: Analyze the cumulative revenue generated over time.

```sql
SELECT
    order_date,
    SUM(revenue) OVER (
        ORDER BY order_date
    ) AS cum_revenue
FROM (
    SELECT
        orders.order_date,
        SUM(order_details.quantity * pizzas.price) AS revenue
    FROM order_details
    JOIN pizzas
        ON order_details.pizza_id = pizzas.pizza_id
    JOIN orders
        ON orders.order_id = order_details.order_id
    GROUP BY orders.order_date
) AS sales;
```

TOTAL ANNUAL REVENUE

$817,860.05

Year-end cumulative total reached on Dec 31, 2015

Q1 Cumulative (Mar 31)

$205,350.00

Mid-Year (Jun 30)

$413,719.75

WHAT IT DOES

First, calculate revenue for each date. Then a window SUM adds each day's revenue to the previous total.

Illustrative SQL example — requires a MySQL version that supports CTEs and window functions.

SQL query spotlight

Question 13 — top 3 pizza types by revenue within each category

Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
SELECT category, name, revenue
FROM (
     SELECT category, name, revenue,
        RANK() OVER(PARTITION BY category ORDER BY revenue DESC) AS rn
     FROM (
          SELECT pizza_types.category, pizza_types.name,
          SUM((order_details.quantity) * pizzas.price) AS revenue
     FROM pizza_types
     JOIN pizzas
        ON pizza_types.pizza_type_id = pizzas.pizza_type_id
     JOIN order_details
        ON order_details.pizza_id = pizzas.pizza_id
     GROUP BY pizza_types.category, pizza_types.name) AS a) AS b
WHERE rn <= 3;
```



Kept exactly as provided in the uploaded project.sql file.

SUM(quantity × price)

Calculates revenue

RANK() OVER

Ranks within each category

WHERE rn <= 3

Keeps the top three ranks

How the ranking query works

1	JOIN TABLES	Connect pizza_types, pizzas, and order_details.
2	CALCULATE REVENUE	Multiply quantity by pizza price, then SUM by category and pizza name.
3	RANK WITHIN CATEGORY	RANK() OVER (PARTITION BY category ORDER BY revenue DESC).
4	FILTER TOP 3	Keep rows where the rank is 3 or below.

Note: RANK() can return more than three rows in a category when there are ties at the cutoff.

What this project analyzes

The 13 questions are grouped into three clear parts:

1. ORDER & PRODUCT DEMAND

Questions 1–5

Total orders, total revenue, highest-priced pizza, most common size, and top 5 pizza types by quantity.

2. CATEGORY & TIME

Questions 6–9

Quantity by category, order distribution by hour, category-wise pizza distribution, and average daily quantity.

3. REVENUE RANKING & TRENDS

Questions 10–13

Top pizzas by revenue, revenue contribution %, cumulative revenue, and top 3 pizza types per category.

In simple words: this project uses SQL to understand what sells, when orders happen, and which pizzas generate revenue.

These are the questions explored by the project; actual numeric findings should be taken from your query results.

Project conclusion

This project demonstrates how SQL can turn raw pizza-order data into structured answers to business questions.

UNDERSTAND DEMAND

Compare order counts, pizza sizes, and popular pizza types.

MEASURE REVENUE

Calculate sales revenue and each pizza type's contribution.

FIND PATTERNS

Explore hourly orders, daily averages, cumulative revenue, and category rankings.

SQL skills practiced

JOIN • GROUP BY • Aggregate functions • ORDER BY • Subqueries • Window functions (RANK)

The presentation describes the analysis performed; add verified result values only when available from your database.

THANK YOU

Pizza Sales Analysis Using SQL

A beginner portfolio project focused on querying data, ranking products, and exploring sales patterns.



SQL • DATA ANALYSIS • INSIGHTS

Nikhil Dhyani

Thank you for viewing my project.